Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет безопасности информационных технологий

Дисциплина:

«Технологии и безопасность блокчейна»

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ

«ImmutableLog»

Выполнили:

Бардышев Артём Антонович,

студент группы N3346

(подпись)

Суханкулиев Мухаммет,

студент группы N3346

(подпись)

Шангина Елизавета Кирилловна,

студентка группы N3346

(подпись)

Проверил:

Федоров Иван Романович,

преподаватель, ФБИТ

(отметка о выполнении)

(подпись)

Санкт-Петербург

2026 г.

СОДЕРЖАНИЕ

Введение 3
1 Архитектура MVP 4
1.1 Смарт-контракт (Solidity) 4
1.2 UI-дашборд (Python/Streamlit) 4
1.3 Системный агент (Python/Web3.py) 4
2 Модель безопасности 6
2.1 Модель нарушителя 6
2.2 Реализованные механизмы защиты 6
2.3 Проактивный анализ уязвимостей (бонус) 6
3 Демонстрация работы прототипа 8
4 Качество и воспроизводимость 10
Заключение 11

ВВЕДЕНИЕ

В современных ИТ-инфраструктурах серверные лог-файлы (auth.log, syslog и др.) являются ключевым источником для расследования инцидентов. При этом классическая проблема ИБ состоит в том, что при компрометации узла (например, при получении злоумышленником root-доступа) журналы могут быть изменены или частично удалены для сокрытия следов атаки. В таком сценарии доверять локально хранимым логам уже нельзя.

Для решения этой проблемы в рамках MVP разработана система ImmutableLog, объединяющая:

– криптографическое хэширование на основе SHA-256;
– запись контрольных значений в блокчейн-реестр;
– пользовательский интерфейс для регистрации и аудита целостности.

Идея системы: хранить не сам лог-файл, а его криптографический отпечаток (хэш), зафиксированный в неизменяемой распределенной среде. Это обеспечивает два критичных свойства:

– Integrity (целостность): любое изменение исходного файла приводит к изменению хэша;
– Immutability (неизменяемость): уже записанные в блокчейн данные не могут быть тихо перезаписаны задним числом.

Таким образом, прототип закрывает базовый риск «подмены истории» в журналировании и закладывает основу для дальнейшего промышленного развития решения.

1 АРХИТЕКТУРА MVP

Архитектура прототипа включает три основных компонента.

1.1 Смарт-контракт (Solidity)

Смарт-контракт (blockchain/immutableLog.sol) выполняет роль защищенного реестра хэшей.

Его функции:

– прием хэша лог-файла и фиксация времени регистрации;

– хранение массива записей аудиторского следа;

– предоставление интерфейса для чтения количества записей и их содержимого.

Роль контракта в архитектуре — быть «источником доверия» для проверки факта и времени регистрации данных, без зависимости от состояния конкретного сервера.

1.2 UI-дашборд (Python/Streamlit)

Компонент ui_app/src/app.py реализует интерфейс ИБ-специалиста:

– загрузка эталонного лог-файла и расчет SHA-256;

– регистрация хэша;

– загрузка файла для аудита и сравнение с эталоном;

– визуальное отображение результата проверки (целостен / скомпрометирован).

UI делает процесс аудита воспроизводимым и удобным для практического использования, в том числе в демонстрационном и учебном режиме.

1.3 Системный агент (Python/Web3.py)

Компонент blockchain/agent.py демонстрирует логику отправки данных в блокчейн через Web3.py.

В целевой архитектуре этот модуль становится системным агентом, который:

– автоматически отслеживает изменения критичных логов;

– периодически рассчитывает хэши;

– отправляет транзакции в смарт-контракт без участия оператора.

На этапе MVP агент реализован как базовый скрипт интеграции и выступает заделом под автоматизацию.

2 МОДЕЛЬ БЕЗОПАСНОСТИ

2.1 Модель нарушителя

Рассматривается нарушитель (внешний атакующий или внутренний инсайдер), который:

– получил повышенные привилегии на сервере (вплоть до root);
– имеет возможность читать/изменять/удалять локальные логи;
– стремится скрыть факт несанкционированного доступа, изменив auth.log.

Цель нарушителя — нарушить достоверность аудиторского следа, чтобы затруднить расследование.

2.2 Реализованные механизмы защиты

1) Криптографическая фиксация состояния файла (SHA-256).

Для каждого эталонного лога вычисляется SHA-256. Даже минимальное изменение (1 символ, удаленная строка, перестановка байтов) приводит к новому хэшу. Это делает незаметную подмену практически невозможной.

2) Вынесение контрольной информации в блокчейн.

Хэш и метаданные регистрации фиксируются в распределенном реестре. После включения транзакции в цепочку злоумышленник не может «тихо» переписать историю аудита так же, как это возможно с локальным файлом на скомпрометированном хосте.

3) Проверка на уровне приложения.

UI при аудите сравнивает текущий хэш проверяемого файла с эталонным значением. Несовпадение интерпретируется как признак компрометации или изменения данных.

Итог: при атаке на сервер злоумышленник может изменить сам auth.log, но не может подделать уже зарегистрированный внешний криптографический след без контроля над блокчейном.

2.3 Проактивный анализ уязвимостей (бонус)

Для смарт-контракта выполнен статический анализ с помощью RemixAI Assistant. Выявлены потенциальные риски (некритичные для MVP, но важные для production):

– отсутствие контроля доступа к registerHash (риск спама ложными записями);
– отсутствие строгой валидации входных данных;
– рост массива записей может повышать стоимость операций.

Важно, что эти риски уже учтены в плане развития проекта (TODO в README.md), где предусмотрено внедрение паттерна Ownable (onlyOwner) и дополнительных проверок входных данных. Это показывает, что проект ведется с позиции secure-by-design, а не только функционального прототипирования.

3 ДЕМОНСТРАЦИЯ РАБОТЫ ПРОТОТИПА

Показан штатный сценарий: пользователь загружает эталонный лог, система рассчитывает SHA-256 и регистрирует запись. Сценарий подтверждает базовую работоспособность контура «UI ↔ блокчейн».

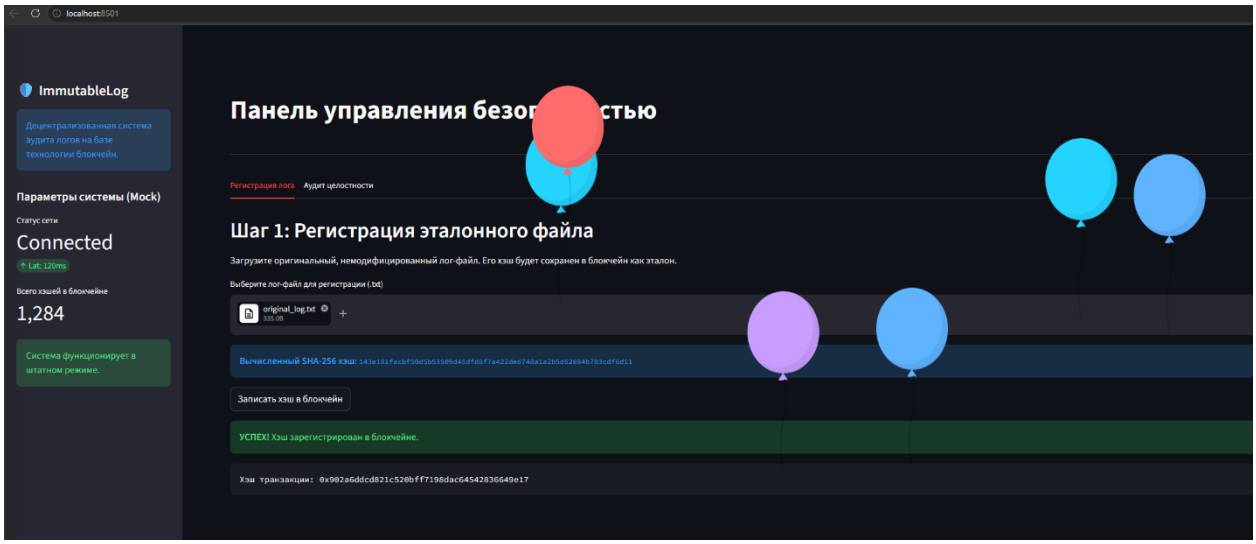


Рисунок 1 - Регистрация хэша эталонного файла.

При проверке неизмененного файла вычисленный хэш совпадает с эталоном. Интерфейс отображает положительный результат аудита.

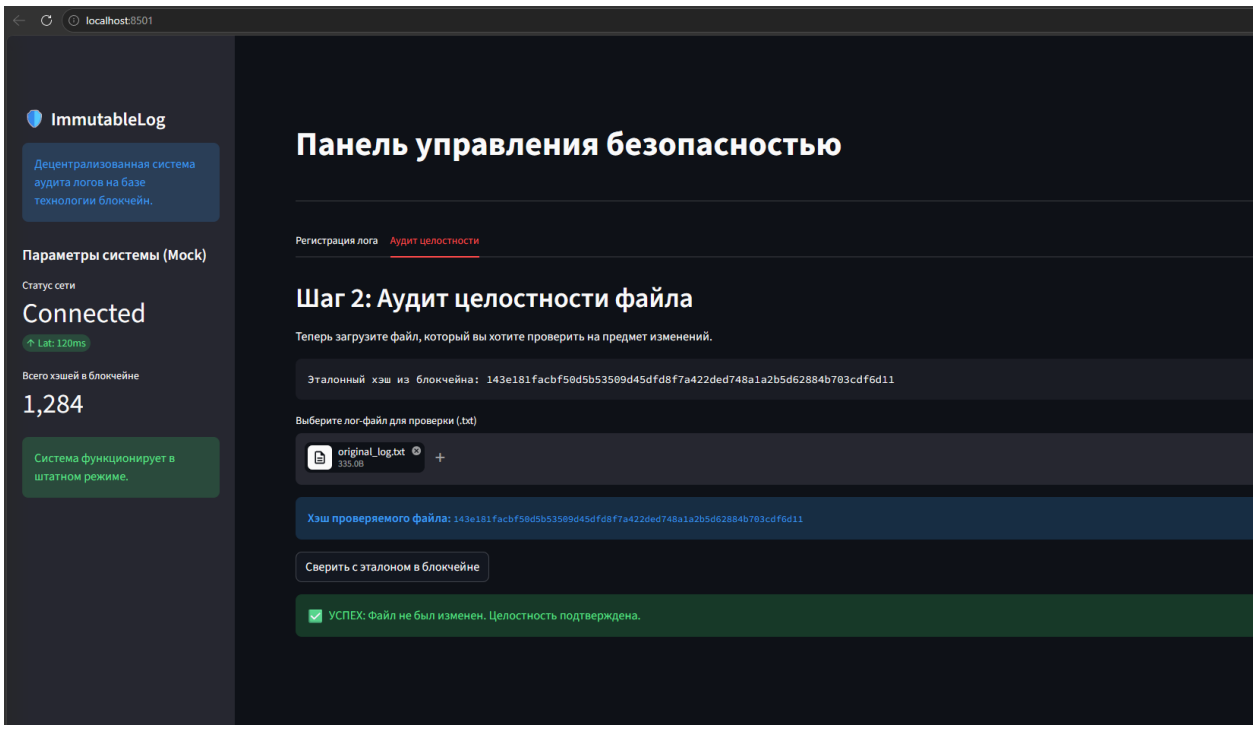


Рисунок 2 - Успешная проверка целостности (штатный режим)

Проверяется модифицированная версия лог-файла; хэш не совпадает с эталоном. UI корректно сигнализирует о нарушении целостности — это ключевой ИБ-кейс проекта.

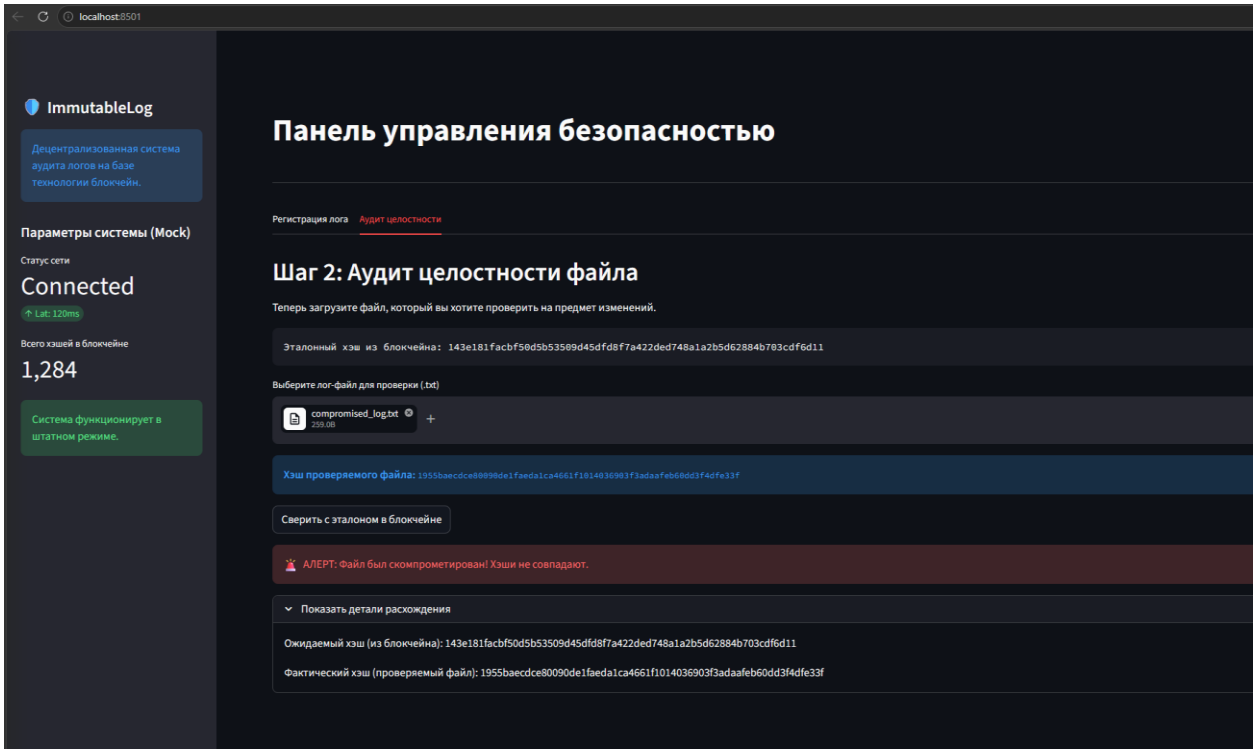
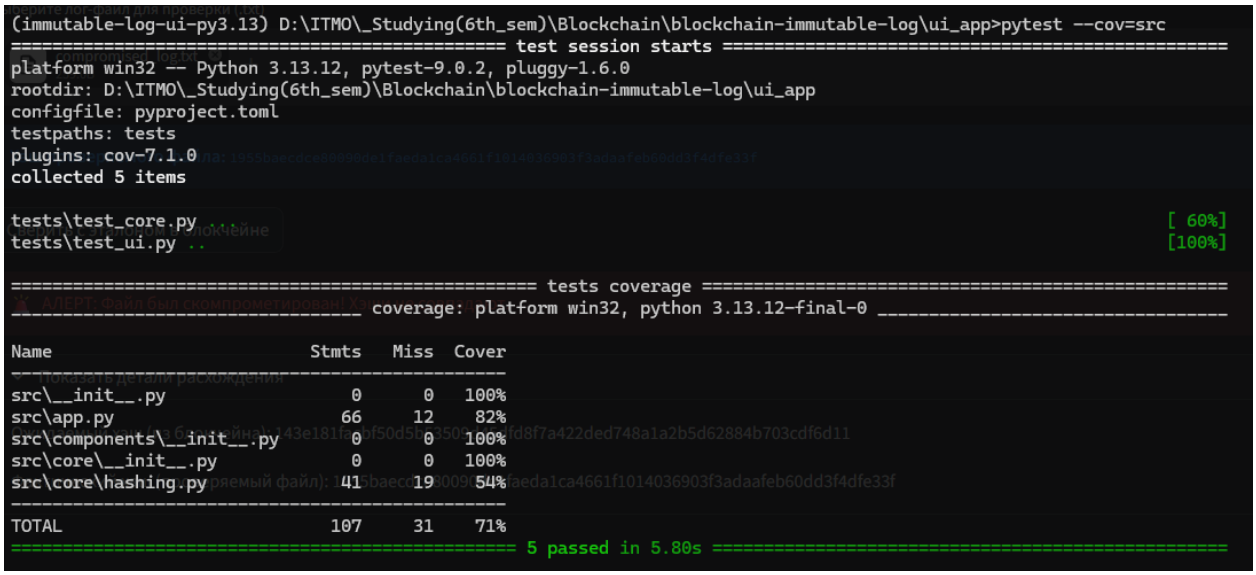


Рисунок 3 - Обнаружение компрометации лога

Показан успешный прогон тестов (pytest) и метрика покрытия, подтверждающая стабильность ключевых сценариев и контроль качества.



```
(immutable-log-ui-py3.13) D:\ITMO\_Studying(6th_sem)\Blockchain\blockchain-immutable-log\ui_app>pytest --cov=src
========================================= test session starts =========================================
platform win32 -- Python 3.13.12, pytest-9.0.2, pluggy-1.6.0
rootdir: D:\ITMO\_Studying(6th_sem)\Blockchain\blockchain-immutable-log\ui_app
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 5 items

tests\test_core.py ...                                                                          [ 60%]
tests\test_ui.py ..                                                                             [100%]

========================================= tests coverage ==========================================
_________________________________ coverage: platform win32, python 3.13.12-final-0 __________________________________

Name                          Stmts   Miss  Cover
-----------------------------------------------------
src\__init__.py                   0      0   100%
src\app.py                       66     12    82%
src\components\__init__.py        0      0   100%
src\core\__init__.py              0      0   100%
src\core\hashing.py              41     19    54%
-----------------------------------------------------
TOTAL                           107     31    71%
========================================= 5 passed in 5.80s ===========================================
```

Рисунок 4 - Результаты автотестов и покрытие кода

4 КАЧЕСТВО И ВОСПРОИЗВОДИМОСТЬ

В MVP применены практики, характерные для промышленной разработки:

– Poetry — управление зависимостями и воспроизводимой Python-средой (pyproject.toml, poetry.lock);

– Pytest + pytest-cov — автоматизированное тестирование ядра и UI-сценариев;

– Docker / Docker Compose — стандартизированный запуск в изолированном окружении;

– модульная структура (core, ui, blockchain) — подготовка к масштабированию и рефакторингу.

Покрытие кода по результатам автотестов: 71% (см. Рис. 4)

ЗАКЛЮЧЕНИЕ

Промежуточный результат: MVP успешно подтверждает жизнеспособность концепции ImmutableLog.

Реализован рабочий контур хэширования и проверки целостности, продемонстрировано обнаружение компрометации файлов, подготовлена интеграционная база для безопасной on-chain фиксации аудиторского следа.

Ближайшие шаги (по TODO из README.md):

Усиление безопасности смарт-контракта:

– внедрение Ownable/onlyOwner;
– валидация входных данных (hash не пустой и корректного формата).

Интеграция реальной on-chain записи из UI:

– замена BlockchainServiceMock на RealBlockchainService;
– перенос логики транзакций из blockchain/agent.py;
– безопасная работа с приватным ключом.

Рефакторинг и поддержка качества:

– актуализация тестов после интеграции реального сервиса;
– мокирование сетевых вызовов в тестах.

Финализация документации и презентации проекта.